

Boring-NixOS

K19G

2026-09-08

Table of contents

Boring NixOS	4
Boring NixOS	5
Who this is for	5
What “boring” means here	5
Roadmap & Syllabus	5
How examples work	7
Nix Fundamentals	8
Nix Fundamentals — How It Actually Works	9
Nix Fundamentals — How It Actually Works	10
Mental model	10
Worked examples	11
Case 1: Inspecting the Nix Store	11
Case 2: Querying Exact Runtime Dependencies	11
Case 3: Evaluating a Pure Expression	11
Case 4: Creating a Trivially Reproducible Derivation	12
Case 5: Garbage Collection and Roots	12
The trap	13
The boring rule	13
Try this	13
Nix Language	14
The Nix Language — From Basics to Advanced Patterns	15
The Nix Language — From Basics to Advanced Patterns	16
Mental model	16
Worked examples	16
Case 1: Types and Attribute Sets	16
Case 2: Functions with Pattern Matching and Default Arguments	17
Case 3: Recursive Attribute Sets (rec)	17
Case 4: Lists, Map, and Filtering	18
Case 5: Importing Files and Modular Abstractions	19
The trap	19
The boring rule	20
Try this	20

Dev Environments	21
Reproducible Development Environments	22
Reproducible Development Environments	23
Mental model	23
Worked examples	23
Case 1: Classic shell.nix with pkgs.mkShell	23
Case 2: Multi-Language Development Stack	24
Case 3: Flake-Based devShell (The Modern Standard)	25
Case 4: Direnv and Automatic Environment Switching	26
The trap	26
The boring rule	27
Try this	27

Boring NixOS

Boring NixOS

From Zero to Production — Reproducible Systems at Scale

A linear guide to writing Nix and managing NixOS systems the way they actually work in production: **deterministic, immutable, and delightfully boring**.

Clarity beats cleverness. Explicit dependencies beat ambient runtime state. The boring default is the configuration that builds identically on your laptop, your CI pipeline, and your production bare-metal cluster six months from now.

Who this is for

- Engineers, platform architects, and SREs who are exhausted by “works on my machine” drift and irreproducible production outages.
- Developers who want clean, instant development environments without container overhead or dependency hell.
- Teams migrating from ad-hoc configuration management (Ansible, bash scripts, Docker Desktop) to pure declarative infrastructure.

You do not need prior Nix or NixOS experience. You only need a terminal, a standard text editor, and a commitment to reproducibility.

What “boring” means here

Nix is a purely functional package manager and domain-specific configuration language. When used with discipline, Nix eliminates entire categories of operational failure: broken library symlinks, partial OS upgrades, conflicting glibc versions, and undocumented server state.

When abused, Nix can become an unreadable puzzle of undocumented overlays and over-engineered macros.

Boring NixOS teaches the disciplined default: clean derivation structures, readable Nix expressions, explicit pinned inputs, and maintainable NixOS modules that scale from a single desktop to fleet infrastructure.

Roadmap & Syllabus

Part	Chapter / Topic	What You Build & Master
Foundations	01: Nix Fundamentals	Store model, <code>/nix/store</code> , cryptographic hashing, dependency graphs, GC
Language	02: The Nix Language	Expressions, let-in bindings, functions, attribute sets, recursive scopes, imports
Environments	03: Reproducible Dev Environments	<code>devShell</code> , <code>shell.nix</code> , multi-language toolchains, Flakes, migrating from Docker Desktop
Packaging	04: Package Management & Overlays	Flakes input system, creating overlays, custom package overrides
Build Systems	05: Building from Source	Derivations, building C/CMake, Rust Cargo, Python, Go, Node.js, and cross-compilation
Operating System	06: NixOS System Configuration	NixOS module system, systemd services, networking, users, firewall, hardware drivers
User Space	07: Home Manager	Declarative dotfiles, shell configs, editor environments, integrating with NixOS
Pipelines	08: CI/CD with Nix	Binary caches (<code>Cachix/Attic</code>), GitHub Actions, GitLab CI, testing, hermetic builds
Containers	09: Containers & Kubernetes	Hermetic container images with <code>dockerTools</code> , Kubernetes manifests, GitOps
IaC	10: Infrastructure as Code	Terraform & OpenTofu provider integration, Ansible provisioning, cloud architectures
Security	11: Secrets & Hardening	SOPS-nix, Age keys, HashiCorp Vault integration, declarative system hardening
Enterprise	12: Platform Engineering at Scale	Internal Developer Platforms (IDPs), monorepo patterns, maintenance and upgrades

How examples work

Every code listing is a self-contained, complete expression or file: - # `default.nix` - # `shell.nix`
- # `flake.nix` - # `configuration.nix`

No fragmented snippets or guessing missing imports. You can copy the listing directly into your file, run the indicated command, and see the exact expected output.

Nix Fundamentals

Nix Fundamentals — How It Actually Works

Nix Fundamentals — How It Actually Works

The traditional operating system model is mutable: packages overwrite shared files in `/usr/lib` and `/usr/bin`, creating unpredictable state and silent breakage. The boring default of Nix is: **every package is an immutable, content-addressed path under `/nix/store`, determined strictly by cryptographic hashes of its inputs.**

Mental model

In traditional package managers (`apt`, `dnf`, `brew`), installing package A can silently upgrade library B, breaking package C that depended on the old version of B. This is the root cause of dependency rot and non-reproducible environments.

Nix solves this by treating software installations like a pure functional evaluation:

1. **The Nix Store (`/nix/store`):** Every artifact lives in an isolated directory whose path includes a cryptographic hash, such as `/nix/store/s69a102...-ripgrep-14.1.0`.
2. **Deterministic Hashes:** The hash is computed from all inputs that produce the package: source code, compiler flags, dependencies, and build scripts. If any dependency changes by a single bit, the output path changes.
3. **No Ambient State:** Builds run inside isolated sandboxes without access to the host network or root filesystem. A program cannot inadvertently link against whatever compiler happened to be installed in `/usr/bin`.
4. **Symlink Trees:** Environments and user profiles are symlink trees pointing to specific `/nix/store` paths. Upgrading or switching environments simply means swapping a symlink.
5. **Garbage Collection:** Deleting a package removes the symlink from your profile. The underlying store paths remain safely until `nix-collect-garbage` cleans unreferenced store entries.

```
/nix/store/  
a1b2c3d4...-glibc-2.39/  
f9e8d7c6...-ripgrep-14.1.0/  -> references glibc-2.39  
b7a6c5d4...-ripgrep-14.0.0/  -> references older glibc
```

Both versions coexist on the same disk without collision or conflict.

Worked examples

Case 1: Inspecting the Nix Store

Check where an installed utility actually lives on disk.

Run:

```
which rg
```

Output:

```
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0/bin/rg
```

Notice that the binary does not reside in `/usr/bin/rg`. It is an immutable, isolated path.

Case 2: Querying Exact Runtime Dependencies

Inspect the dependency graph of an executable using `nix-store -q --references`.

Run:

```
nix-store -q --references $(which rg)
```

Output:

```
/nix/store/7r44p12l4b72j9m9l09k96yghz02f8n7-glibc-2.39  
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

Every library needed by `rg` is explicitly recorded. There are no undeclared ambient dependencies.

Case 3: Evaluating a Pure Expression

Save as `hello.nix`. A minimal Nix expression that defines a string value.

```
# hello.nix  
let  
  greeting = "Hello, reproducible world!";  
in  
  greeting
```

Run:

```
nix-instantiate --eval hello.nix
```

Output:

```
"Hello, reproducible world!"
```

Nix is an expression language. Evaluating an expression computes a deterministic value.

Case 4: Creating a Trivially Reproducible Derivation

Save as `greeting.nix`. A complete self-contained derivation using the built-in `derivation` primitive.

```
# greeting.nix
derivation {
  name = "desk-greeting";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    "echo 'Desk system operational' > $out"
  ];
}
```

Run:

```
nix-build greeting.nix
```

Output:

```
these 1 derivations will be built:
  /nix/store/m6a1p...-desk-greeting.drv
building '/nix/store/m6a1p...-desk-greeting.drv'...
/nix/store/k3b29...-desk-greeting
```

Inspect the generated output:

```
cat result
```

Output:

```
Desk system operational
```

`result` is a local symlink pointing directly into `/nix/store`.

Case 5: Garbage Collection and Roots

Remove the symlink and run garbage collection.

Run:

```
rm result
nix-collect-garbage
```

Output:

```
finding garbage collector roots...
deleting garbage...
deleting '/nix/store/k3b29...-desk-greeting'
1 store paths deleted, 0.00 MiB freed
```

Because no active symlink (garbage collector root) references **desk-greeting**, the store reclaims the space.

The trap

Save as `bad_build.sh`. Trying to run conventional `make install` or `pip install` globally directly writes into system directories, creating untracked file clutter.

```
# bad_build.sh
# Anti-pattern: mutating shared system state
sudo cp myapp /usr/local/bin/myapp
```

When `/usr/local/bin/myapp` breaks after an OS update, no tool knows which dependencies it relied upon or how to roll it back.

The fix: Define software as a Nix derivation or shell environment. Every file remains tracked, sandboxed, and reversible.

The boring rule

- All binaries and libraries belong in `/nix/store`, never in mutable system paths.
- Depend on explicit store paths, never on whatever happens to be on `$PATH`.
- Use `nix-store -q --tree` to inspect dependency chains.
- Run `nix-collect-garbage -d` periodically to remove stale generations and unreferenced store paths.

Try this

1. Run `nix-store -q --tree $(which nix)` and observe the complete tree of dependencies required by the Nix binary itself.
2. In `greeting.nix`, modify the greeting text to `'Front desk open'` and re-run `nix-build`. Note how the hash in the resulting `/nix/store` path changes.
3. Check your current garbage collector roots using `nix-store --gc --print-roots`.

Nix Language

The Nix Language — From Basics to Advanced Patterns

The Nix Language — From Basics to Advanced Patterns

Nix is not a general-purpose programming language; it is a **pure, lazy, functional configuration language**. The boring default is: **write explicit attribute sets with clear argument destructuring, keep expressions small, and let laziness compute only what is actually built**.

Mental model

Nix has six primary primitive types (integers, floats, booleans, strings, paths, and null) plus lists and attribute sets.

1. **Pure:** Functions have no side effects. A function called with the same inputs always produces identical output.
2. **Lazy:** Values are only evaluated when their result is needed. You can define infinite lists or huge attribute sets without paying a performance penalty until a specific key is accessed.
3. **Attribute Sets** (`{ a = 1; b = 2; }`): The primary dictionary/object construct.
4. **Functions** (`x: x + 1`): All functions take exactly one argument. Multi-argument functions are curried (`x: y: x + y`) or accept an attribute set (`{ x, y }: x + y`).
5. **Let-In** (`let ... in ...`): Defines local variables scoped strictly within the expression following in.

Worked examples

Case 1: Types and Attribute Sets

Save as `types.nix`. Explore integers, strings, string interpolation, and attribute sets.

```
# types.nix
let
  deskName = "Central Terminal";
  port = 8080;
  deskConfig = {
    name = deskName;
    listen = "127.0.0.1:${toString port}";
    enabled = true;
    tables = [ 1 2 3 4 5 ];
```



```
};
in
deskConfig
```

Run:

```
nix-instantiate --eval --strict types.nix
```

Output:

```
{ enabled = true; listen = "127.0.0.1:8080"; name = "Central Terminal"; tables = [ 1 2 3 4 5 ];
```

`--strict` forces evaluation of all nested fields inside the attribute set.

Case 2: Functions with Pattern Matching and Default Arguments

Save as `functions.nix`. Create a service generator function with default values.

```
# functions.nix
let
  makeService = { name, port ? 3000, debug ? false }: {
    serviceName = "desk-${name}";
    listenPort = port;
    environment = if debug then "development" else "production";
  };
in
{
  defaultService = makeService { name = "orders"; };
  customService = makeService { name = "billing"; port = 9000; debug = true; };
}
```

Run:

```
nix-instantiate --eval --strict functions.nix
```

Output:

```
{ customService = { environment = "development"; listenPort = 9000; serviceName = "desk-billing";
```

`port ? 3000` provides a fallback default if the caller does not supply that attribute.

Case 3: Recursive Attribute Sets (rec)

Save as `recursive.nix`. An attribute set where fields refer to neighboring fields.

```
# recursive.nix
let
  cluster = rec {
    domain = "internal.desk";
    apiHost = "api.${domain}";
    authHost = "auth.${domain}";
    serviceEndpoints = [ apiHost authHost ];
  };
in
cluster
```

Run:

```
nix-instantiate --eval --strict recursive.nix
```

Output:

```
{ apiHost = "api.internal.desk"; authHost = "auth.internal.desk"; domain = "internal.desk"; ser
```

The `rec` keyword allows sibling fields to resolve one another.

Case 4: Lists, Map, and Filtering

Save as `collections.nix`. Filter and transform a list of table IDs.

```
# collections.nix
let
  tables = [ 1 2 3 4 5 6 7 8 9 10 ];
  isEven = n: (n / 2) * 2 == n;
  evenTables = builtins.filter isEven tables;
  tableLabels = map (n: "Table-${toString n}") evenTables;
in
tableLabels
```

Run:

```
nix-instantiate --eval --strict collections.nix
```

Output:

```
[ "Table-2" "Table-4" "Table-6" "Table-8" "Table-10" ]
```

`builtins.filter` and `map` are standard functional utilities available without external packages.

Case 5: Importing Files and Modular Abstractions

Split logic across separate files using `import`.

Save as `config.nix`:

```
# config.nix
{
  domain = "desk.local";
  defaultTimeoutSeconds = 30;
}
```

Save as `main.nix`:

```
# main.nix
let
  cfg = import ./config.nix;
in
{
  serverHost = "auth.${cfg.domain}";
  timeout = cfg.defaultTimeoutSeconds;
}
```

Run:

```
nix-instantiate --eval --strict main.nix
```

Output:

```
{ serverHost = "auth.desk.local"; timeout = 30; }
```

`import ./file.nix` parses and evaluates the target file, returning its evaluated expression.

The trap

Save as `infinite_rec.nix`. Overusing `rec` creates circular dependencies that cause infinite loops during evaluation.

```
# infinite_rec.nix
# Anti-pattern: circular reference inside rec
rec {
  a = b + 1;
  b = a + 1;
}
```

Run:

```
nix-instantiate --eval --strict infinite_rec.nix
```

Output:

```
error: infinite recursion encountered
```

The fix: Avoid `rec` when possible. Prefer clean, explicit `let ... in` bindings where dependency order is obvious and acyclic.

The boring rule

- Prefer explicit `let ... in` blocks over `rec` attribute sets.
- Use attribute set destructuring `{ pkgs, lib, ... }:` for clean component interfaces.
- Always use `? default` when an option has a sane default value.
- Keep `.nix` files small and single-purpose; compose them with `import`.
- Rely on laziness: define full configurations without worrying about premature evaluation.

Try this

1. In `functions.nix`, add an optional boolean argument `ssl ? true` and include `"https"` or `"http"` in the service URL based on its value.
2. In `collections.nix`, write a function that sums all numbers in the list.
3. Create a helper file `math.nix` exporting `add` and `multiply`, and call it from a new `app.nix`.

Dev Environments

Reproducible Development Environments

Reproducible Development Environments

Development environment drift is the single most common source of onboarding friction and subtle bugs across software teams. The boring default is: **a single `flake.nix` or `shell.nix` checked into the repository root that provisions exact, pinned compilers, tools, and linters instantly without VM overhead.**

Mental model

Teams traditionally manage development environments with one of three flawed approaches: - **Global system packages:** “Install Go with brew and Python with apt” leads to subtle version mismatches across developer laptops. - **Version managers (`asdf`, `pyenv`, `nvm`):** Better, but only manage language runtimes. They fail to manage underlying C libraries, headers, `pkg-config`, or system tools like `cmake` and `jq`. - **Heavy Docker containers (Docker Desktop):** Slow to start, consume excessive memory/CPU, and introduce file permission headaches between container and host filesystems.

A Nix `devShell` provides an instant ephemeral shell environment where the exact toolchain and dependencies are injected directly into your `$PATH` and library flags without running a virtual machine or container daemon.

When you exit the shell, your system remains pristine.

Host OS (Linux / macOS)

`nix develop`

Injects exact Go 1.27, Node 22, Terraform, and `libpq`

Sets `$PKG_CONFIG_PATH` and compiler flags

Exiting drops back to your normal pristine host environment

Worked examples

Case 1: Classic `shell.nix` with `pkgs.mkShell`

Save as `shell.nix`. A classic development shell that provides `ripgrep`, `jq`, and `curl`.

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
```

```

    pkgs.ripgrep
    pkgs.jq
    pkgs.curl
];

shellHook = ''
    echo "=== Desk Operations Environment Ready ==="
    echo "Available: $(rg --version | head -n 1)"
'';
}

```

Run:

```
nix-shell --run "echo Shell active"
```

Output:

```

=== Desk Operations Environment Ready ===
Available: ripgrep 14.1.0
Shell active

```

Running `nix-shell` drops you into a subshell with those exact utilities on `$PATH`. Exiting returns to your system shell.

Case 2: Multi-Language Development Stack

Save as `multi_lang_shell.nix`. A comprehensive desk service environment requiring Go, Python, and PostgreSQL tools.

```

# multi_lang_shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.go
    pkgs.gopls
    (pkgs.python3.withPackages (ps: [
      ps.requests
      ps.pydantic
    ]))
    pkgs.postgresql
  ];

  env = {
    CGO_ENABLED = "0";
    PGHOST = "127.0.0.1";
  };
}

```



```
}
```

Run:

```
nix-shell multi_lang_shell.nix --run "go version && python3 --version"
```

Output:

```
go version go1.27.1 linux/amd64
Python 3.12.7
```

Python packages and Go compiler are co-located in this shell without virtualenv activation scripts or Docker compose setups.

Case 3: Flake-Based devShell (The Modern Standard)

Save as `flake.nix`. The contemporary, fully pinned, and lockfile-backed approach to dev environments.

```
# flake.nix
{
  description = "Desk management service development shell";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
  };

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs { inherit system; };
    in
    {
      devShells.${system}.default = pkgs.mkShell {
        packages = [
          pkgs.git
          pkgs.nixpkgs-fmt
          pkgs.gh
        ];

        shellHook = ''
          export REPO_ROOT=$(pwd)
          echo "Desk Flake devShell loaded."
        '';
      };
    };
};
```

```
}
```

Run:

```
nix develop --command sh -c "echo \${REPO_ROOT} && echo \${Desk Flake devShell loaded}"
```

Output:

```
Desk Flake devShell loaded.  
/home/user/project
```

`flake.lock` automatically pins the exact commit of `nixpkgs`, ensuring that every teammate receives identical software down to the byte.

Case 4: Direnv and Automatic Environment Switching

Developers should not need to remember to run `nix develop` every time they change directories. Combine Nix with `direnv`.

Save as `.envrc`:

```
# .envrc  
use flake
```

Run:

```
direnv allow
```

Output:

```
direnv: loading .envrc  
direnv: using flake  
direnv: export +REPO_ROOT ~PATH
```

Whenever you `cd` into the project directory, your editor and shell automatically load all project tools. When you `cd` out, your environment unloads them.

The trap

Save as `docker_drift.txt`. Relying on bulky Docker containers for routine local development causes synchronization issues and high overhead.

```
# Anti-pattern: Running basic compilers inside fat containers  
docker run -v $(pwd):/app -it ubuntu:latest bash -c "apt-get update && apt-get install -y gcc"
```

Problems: - 10x slower file I/O on macOS/Windows host mounts. - Host IDE cannot easily see language servers (`gopls`, `pyright`) running inside the container. - High RAM consumption compared to native execution.

The fix: Use Nix `devShell` for native host performance, direct editor integration, and zero container overhead.

The boring rule

- Commit a `flake.nix` and `flake.lock` at the root of every shared repository.
- Use `pkgs.mkShell` with explicit `packages` lists.
- Set shared environment variables (`CGO_ENABLED`, `PGDATABASE`) in the shell definition.
- Pair with `direnv` and use `flake` for zero-click environment activation.
- Never force developers to install language runtimes globally on their workstations.

Try this

1. In `flake.nix`, add `pkgs.htop` to the packages list and run `nix develop --command htop` to verify on-demand binary injection.
2. Add a `shellHook` script that prints an ASCII banner when entering the shell.
3. Test that variables exported in `shell.nix` vanish as soon as you exit the nix-shell.